AIST4010: Foundation of Applied Deep Learning
Assignment 2

Name: JIANG Huanyun Student ID: 1155233165

Question 1 (10 pts)

Gradient vanishing: by the chain rule, the gradient of the loss with respect to an early-layer weight is a product of many per-layer terms. If each term shrinks the signal (for example the derivative of sigmoid/tanh is at most 0.25, or the weight matrices are repeatedly small), the product decays exponentially with depth, the gradients reaching early layers become almost zero, and those layers stop learning.

Gradient explosion: if the per-layer terms amplify the signal (for example large weight norms), the product grows exponentially with depth, the gradients become NaN/Inf, and training diverges.

Solutions:

1. Use ReLU (or Leaky ReLU/ELU) instead of sigmoid/tanh. Its derivative is 1 for positive inputs, so the signal is no longer shrunk by a factor of at most 0.25 at every layer.
2. Use proper weight initialization (Xavier for tanh/sigmoid, He for ReLU), which keeps the variance of activations and gradients constant across layers, so the per-layer factor stays near 1.
3. Add normalization layers (BatchNorm/LayerNorm), which keep the signal in the non-saturating regime and stop it from decaying or blowing up.
4. Clip the gradient norm to a threshold; this is the standard fix for gradient explosion.
5. Use residual (skip) connections or LSTM/GRU gates, which provide paths where the gradient can flow without repeated multiplication.

Question 2 (5 pts)

Similarities:

Both slide a learnable kernel over the input and compute a weighted sum of a local neighborhood at every position. Both have local connectivity and weight sharing, which give translation equivariance and few parameters. Both are stacked with nonlinearities and pooling to learn hierarchical features, and both follow the same output size formula $(n + 2p - k)/s + 1$.

Differences:

A 2D convolution kernel is $k_h \times k_w$ and slides over two spatial axes (height and width), so it is used for static images and only captures spatial patterns.

A 3D convolution kernel is $k_d \times k_h \times k_w$ and additionally slides over a depth axis (time for video, or the third dimension for CT/MRI volumes), so it also captures motion and 3D structure. For the same kernel size, a 3D convolution has about k times more parameters and computation and overfits more easily. Because of this cost, video is often processed by 2D convolution plus temporal fusion (e.g. RNN or (2+1)D decomposition) as a cheaper approximation of 3D convolution.

Question 3 (10 pts)

Possible reasons:

1. Learning rate too large: the updates overshoot the minimum, the loss oscillates or diverges and may become NaN.
2. Learning rate too small: the loss decreases too slowly to be visible in the first few epochs.
3. Bad initialization: all-zero (or identical) weights keep all neurons symmetric so they never learn; very large weights saturate sigmoid/tanh units so their gradients are almost zero.
4. Vanishing gradients in a deep network with sigmoid/tanh activations (see Q1).
5. Output activation does not match the loss (e.g. sigmoid output with cross-entropy for multi-class classification).
6. Data problems: inputs not normalized (very different scales), or labels misaligned with inputs.
7. The model has too little capacity, or there is a bug in the training loop (gradients not zeroed, optimizer not attached to the parameters, etc.).

Question 4 (10 pts)

How imbalance affects training and predictions:

The training loss is dominated by the majority class, so the decision boundary is biased toward it and minority samples contribute little to the gradients. The model may simply predict the majority class everywhere, which still gives high accuracy, so accuracy alone hides the failure. With few minority samples the model cannot learn their true distribution, and minority-class predictions become noisy and unreliable.

Ways to alleviate:

1. Resampling: oversample the minority class (e.g. SMOTE) or undersample the majority class.
2. Class-weighted loss: weight each class inversely to its frequency (weighted cross-entropy), or use the focal loss which also down-weights easy examples.
3. Balanced batch sampling: make every mini-batch contain roughly equal numbers of samples from each class.
4. Data augmentation for the minority classes to enlarge their effective sample size.
5. Evaluate with per-class metrics (F1, AUC, balanced accuracy) and tune the decision threshold instead of the default 0.5.

Question 5 (35 pts)

Setup:

The classic LeNet-5: Conv(1 to 6, 5x5, pad 2), Tanh, AvgPool 2x2, Conv(6 to 16, 5x5), Tanh, AvgPool 2x2, FC 120, FC 84, FC 10. All runs use SGD with learning rate 0.01, batch size 128, 10 epochs, cross-entropy loss, normalized Fashion-MNIST images, no data augmentation, a fixed random seed and PyTorch default initialization, so each experiment changes only one factor.

Baseline test accuracy: 80.75%.

Experiments (test accuracy of each setting; every factor is changed alone):

(a) Conv layers:
remove one conv layer: 81.12%;
add one conv layer (6-16-32): 82.30%.

(b) FC layers:
remove one FC layer: 80.85%;
add one FC layer (120-84-42): 79.43%.

(c) Dropout:
dropout 0.5: 79.46%;
dropout 0.25: 80.34%.

(d) Pooling:
MaxPool 2x2 instead of AvgPool: 83.28%;
AvgPool with kernel 3: 77.81%.

(e) Conv window size:
3x3 kernels: 79.45%;
7x7 kernels: 81.06%.

(f) Number of channels:
half of the channels: 80.87%;
double the channels: 81.93%.

(g) Activation function:
sigmoid: 10.00% (fails to learn);
ReLU: 81.53%.

Best result: combining MaxPool, doubled conv channels and ReLU gives 84.48% test accuracy (+3.73% over the baseline).

Observations:

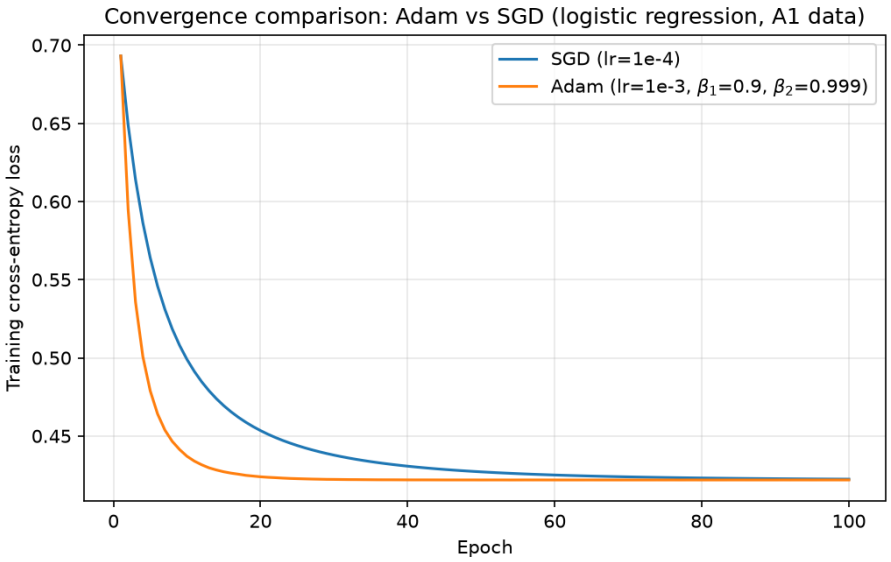
MaxPool helps most because passing the strongest local response fits clothing textures and edges better than averaging. Wider conv layers and ReLU also help; ReLU converges faster and avoids saturation. Sigmoid fails completely because stacking many sigmoids makes the gradients vanish (this is exactly Q1). Dropout slightly hurts because with only 10 epochs the network has not yet overfit. Kernel 7x7 is slightly better than 5x5 while 3x3 is worse. Adding FC layers overfits.

Question 6 (10 pts)

I implemented Adam in NumPy (learning rate 10^{-3}, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, mini-batch size 32) and applied it to the logistic regression problem from A1. SGD uses learning rate 10^{-4} as required. Both start from $W = 0$, $b = 0$ with the same seed and run 100 epochs.

Result:

Adam converges much faster than SGD. At epoch 3 the training loss of Adam is 0.536 while SGD is at 0.614; at epoch 10 it is 0.437 versus 0.499. The final training loss is 0.4219 for Adam and 0.4224 for SGD (test accuracy 84.0% versus 83.0%). The speed comes from the 10 times larger default learning rate, the per-parameter adaptive step sizes (second moment) and the momentum (first moment); the final quality of the two optimizers is almost the same.



Question 7 (20 pts)

Efficient Estimation of Word Representations in Vector Space

Motivation: previous neural language models were accurate but very expensive to train on large corpora, which limited their scalability.

Novelty: the hidden layer is removed and two simple log-linear models are proposed, CBOW and Skip-gram, trained with hierarchical softmax and negative sampling, so training becomes much cheaper (word2vec).

Intuition: words with similar contexts have similar meanings, so predicting the context forces the learned vectors to encode semantic and syntactic relations; the embeddings even support analogies such as King - Man + Woman is close to Queen.

Implementation: the official word2vec code is in the dav/word2vec and tmikolov/word2vec repositories on GitHub (`https://github.com/dav/word2vec`, `https://github.com/tmikolov/word2vec`).

Attention Is All You Need

Motivation: RNN/LSTM sequence models are sequential, hard to parallelize, and lose long-range information through many recurrent steps.

Novelty: recurrence is removed entirely; the Transformer uses only multi-head scaled dot-product self-attention and feed-forward layers with residual connections and layer normalization.

Intuition: attention lets every token directly look at every other token and weight its relevance, so information can travel between distant positions in a single step and the whole sequence can

be processed in parallel.

Implementation: the official code is in the tensorflow/tensor2tensor repository on GitHub (`https://github.com/tensorflow/tensor2tensor`); a widely used implementation is in the huggingface/transformers repository (`https://github.com/huggingface/transformers`).

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Motivation: standard language models were unidirectional, which limited the representations for tasks that need context from both sides.

Novelty: BERT pre-trains deep bidirectional Transformers with masked language modeling and next sentence prediction, then fine-tunes the same model with a small output layer, reaching state of the art on 11 NLP tasks.

Intuition: masking tokens forces the model to use both the left and right context, giving richer representations; pre-training first captures general language knowledge that is then transferred to downstream tasks.

Implementation: the official code is in the google-research/bert repository on GitHub (`https://github.com/google-research/bert`).

MSA Transformer

Motivation: extracting evolutionary information from protein multiple sequence alignments previously required expensive iterative procedures.

Novelty: the model applies attention along the rows (sequences) and the columns (positions) of an MSA, and is pre-trained with masked language modeling on 26 million MSAs.

Intuition: in an MSA, columns are aligned positions and rows are homologous sequences; column attention learns which positions co-vary (contacting residues co-evolve) and row attention learns which sequences are informative, so the model recovers structural information without explicit supervision.

Implementation: the official code is in the facebookresearch/esm repository on GitHub (`https://github.com/facebookresearch/esm`).